

Compute, Storage, Networking and the Deployment Landscape

A working map of Google Cloud for the service you are about to ship

WHAT WE'LL COVER

Seven chapters, five labs

01

Landing in the sandbox

02

The compute spectrum, and choosing

03

Compute Engine and instance groups

04

Google Kubernetes Engine

05

Cloud Functions and event-driven compute

06

Storage and databases

07

Networking: reaching and protecting the service

01

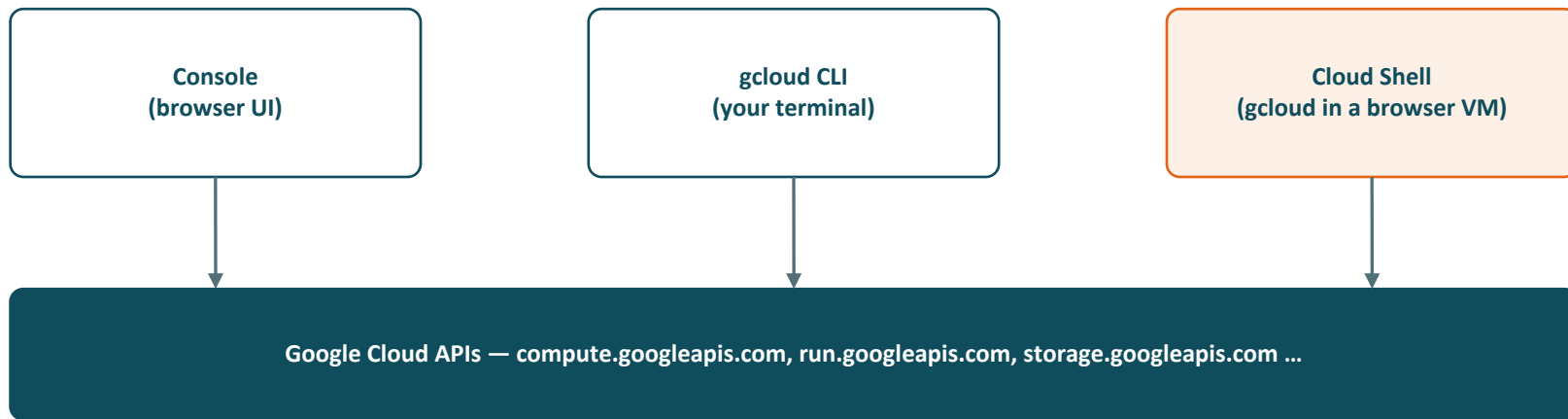
Landing in the sandbox

Open Cloud Shell, point it at your sandbox project, and prove every tool we need today is there.

- 01** Landing in the sandbox
- 02 The compute spectrum, and choosing
- 03 Compute Engine and instance groups
- 04 Google Kubernetes Engine
- 05 Cloud Functions and event-driven compute
- 06 Storage and databases
- 07 Networking: reaching and protecting the service

THREE DOORS, ONE API

Console, Cloud Shell and gcloud talk to the same control plane

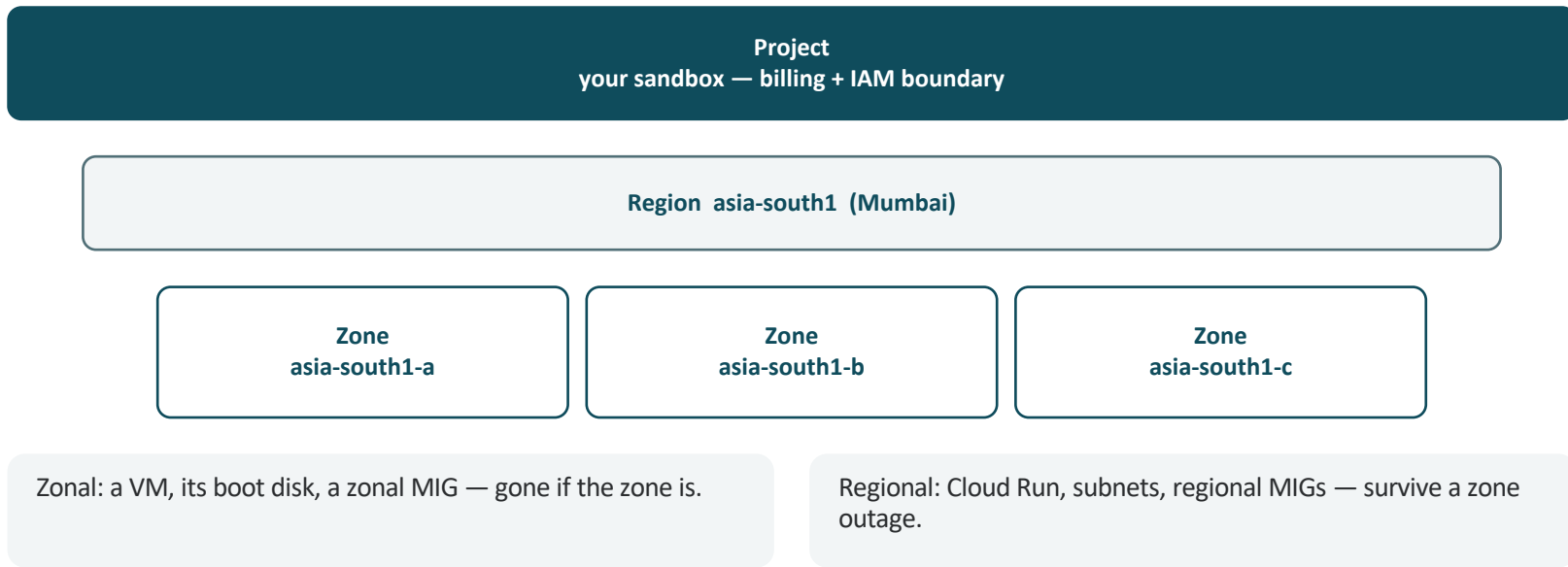


Same identity (your Google account), same permissions (IAM), same audit log. Cloud Shell is the door we use today: nothing to install, one tool version for the whole room, ~5 GB of home directory that persists between sessions.

THE RULE If you can click it, you can script it — learn the gcloud form, the console will always be there.

PROJECT, REGION, ZONE

Every resource has an address: project → region → zone



THE RULE

Decide the region once, early — data, VMs and services in different regions pay latency and egress.

Set the project, enable the APIs, check the tools

HANDS-ON · LAB 0

1

Open the console, pick your sandbox project, click the Cloud Shell icon

console.cloud.google.com → icon top right; the prompt ends with (your-project-id)

2

Point gcloud at the project and the region

gcloud config set project YOUR_PROJECT_ID · gcloud config set compute/region asia-south1 ·
gcloud config set compute/zone asia-south1-a

3

Enable the six APIs used today (one command)

gcloud services enable compute.googleapis.com run.googleapis.com cloudbuild.googleapis.com
artifactregistry.googleapis.com secretmanager.googleapis.com storage.googleapis.com

4

Verify the toolchain

gcloud --version | head -1 · docker --version · python3 --version · kubectl version --client |
head -1

Console path: APIs & Services → Enable APIs and services → search each API → Enable. Nothing here costs money; if a step fails, raise a hand.

EXPECTED

```
$ gcloud config set project sandbox-rohan-07
Updated property [core/project].
$ gcloud services enable
compute.googleapis.com \
  run.googleapis.com \
  cloudbuild.googleapis.com \
  artifactregistry.googleapis.com \
  secretmanager.googleapis.com \
  storage.googleapis.com
Operation "operations/acat.p2-6..." finished
successfully.
$ gcloud --version | head -1
Google Cloud SDK 583.0.0
$ python3 --version
Python 3.13.5
```

THE RULE

One project, one region, six APIs — check the prompt shows your project id before every lab.

02

The compute spectrum, and choosing

Four ways to run the same code, what each costs you to operate, and the six criteria that actually decide between them.

- 01 Landing in the sandbox
- 02 The compute spectrum, and choosing**
- 03 Compute Engine and instance groups
- 04 Google Kubernetes Engine
- 05 Cloud Functions and event-driven compute
- 06 Storage and databases
- 07 Networking: reaching and protecting the service

THE COMPUTE SPECTRUM

Four ways to run the same code — control traded for convenience

Virtual machine

Compute Engine

You own: OS, patches, runtime, process, scaling

Google owns: hardware, hypervisor, network

Container

Cloud Run / GKE

You own: image, app config

Google owns: OS, runtime host, scheduling

Kubernetes

GKE

You own: manifests, cluster shape (Standard) or just workloads (Autopilot)

Google owns: control plane, node upgrades

Serverless

Cloud Run, Cloud Run functions

You own: code + a container or a function

Google owns: everything else, incl. scale-to-zero

more control, more to operate

less to operate, fewer knobs

THE RULE

Start as far right as the workload allows; move left only for a reason you can name.

WHAT "MANAGED" MEANS

The same web app, four ways — what is left on your plate

Responsibility	VM (Compute Engine)	Container (Cloud Run)	Kubernetes (GKE Autopilot)	Function (Cloud Run functions)
OS patching	you	Google	Google	Google
Runtime install (Python, Java...)	you	you (image) / buildpacks	you (image)	Google (base image)
Process supervision, restarts	you (systemd)	Google	Google (kubelet)	Google
HTTPS endpoint + certificate	you (LB + cert)	Google (run.app URL)	you (Ingress / Gateway)	Google
Scaling	you (MIG + autoscaler)	Google (per request)	Google (HPA you configure)	Google (per event)
Scale to zero	no	yes	no (Autopilot bills per pod)	yes
Logs + metrics	you (install agent)	Google	Google	Google

THE RULE

Count the "you" cells in a column — that is the operations team you are hiring, whether or not you meant to.

Where would you put it?

Same generic company, four workloads. Pick a rung for each and be ready to say why.

A) Nightly report

Runs once a day for 20 minutes, reads the database, writes a PDF to a bucket.

B) Public API, 24×7

Steady 200 req/s, spiky at 9 am, must answer in < 300 ms.

C) Licensed legacy app

Vendor binary, needs a specific kernel module and a static IP for its licence server.

D) Image-resize on upload

Fires when a file lands in a bucket; 50 ms of work; thousands of times a day, zero at night.

What you give up moving right — and why that is usually fine

MOVING RIGHT COSTS

- Cold starts when scaling from zero (hundreds of ms to seconds)
- Request/response model: long-lived connections and background threads need care
- Ephemeral local disk — state must live elsewhere (bucket, database)
- Networking is opt-in: no VPC by default, egress via Direct VPC egress when needed
- Fewer knobs: no kernel modules, no custom OS agents

MOVING RIGHT BUYS

- Zero patching, zero process supervision, zero capacity planning
- Scale to zero: an idle service costs nothing
- HTTPS, revisions, rollbacks and traffic splitting built in
- Logs and metrics with no agent to install
- One deploy command from source to URL

THE RULE

A stateless HTTP service with a managed database is the common case — and the right edge was built for exactly that.

CHOOSING BETWEEN THEM

Six criteria that actually decide it — not the feature list

Criterion	Points toward VM (Compute Engine)	Points toward GKE	Points toward Cloud Run / functions
1 • State	State on local disk, long-lived sockets, licensed daemons	Stateful sets with volumes, many cooperating services	Stateless request handlers; state in a DB or bucket
2 • Traffic shape	Flat, predictable 24×7 (reserve and save)	Mixed fleet sharing capacity	Bursty, idle at night, unknown
3 • Latency budget	Always warm by definition	Always warm; you tune	Fine with min-instances; cold starts otherwise
4 • Control needed	Kernel, drivers, custom OS agents	Sidecars, custom networking, GPUs	None beyond env, CPU, memory, concurrency
5 • Team	Sysadmin skills, runbooks exist	Kubernetes fluency, platform team	Developers only; no ops team
6 • Portability	Image formats differ per cloud	Manifests port between clusters	Container ports anywhere; triggers do not

THE RULE

Answer the six honestly for the workload in front of you — the column with the most ticks is the decision.

CHOOSING BETWEEN THEM

Two anti-patterns you will be tempted by — and the plain version

ANTI-PATTERN 1

- A GKE cluster for one stateless API — "we might need Kubernetes later"
- Cost: upgrades, RBAC, networking, YAML — a platform team for one service

ANTI-PATTERN 2

- A VM for a webhook that runs 50 ms once an hour — "I know how to do it on a VM"
- Cost: 24×7 bill, patching, a process to supervise, no scale-to-zero

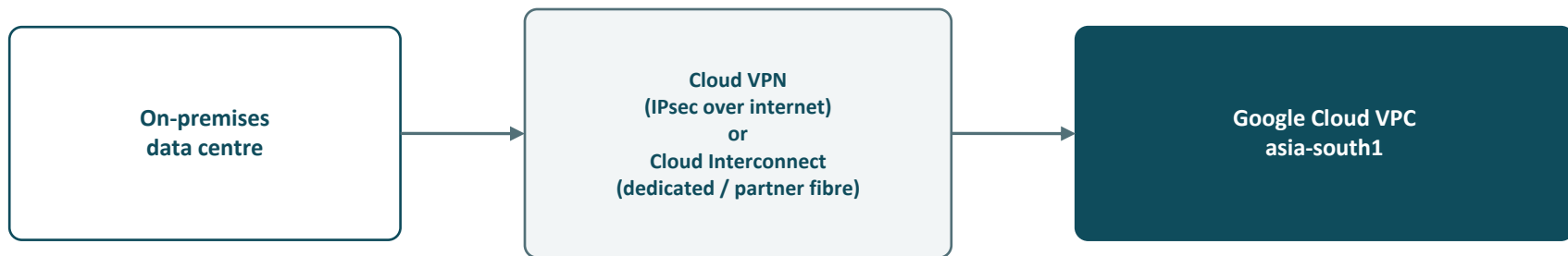
THE PLAIN VERSION (GENERIC WEB APP)

- Front end and API → Cloud Run services · nightly batch → Cloud Run job · on-upload resize → Cloud Run function
- Licensed reporting engine that needs a kernel module → the one Compute Engine VM, in a MIG of 1 for autohealing
- A cluster only when the count of services and the team's fluency both say so

THE RULE

Familiarity and ambition are not criteria — if the six say Cloud Run, the cluster can wait.

What they are — and the toolkit that connects them



GKE Enterprise: one control plane for clusters on-prem, on GCP, on other clouds (fleet, config, policy)

Legitimate reasons: data residency / regulation · an existing data centre with life left · vendor-risk policy · an acquisition on another cloud. "Avoid lock-in" alone is rarely one — see the bill on the next slide.

THE RULE

Hybrid is a network and an operating model, not a feature — budget for both before the first packet.

What they cost you — the five bills nobody puts on a slide

Bill	What it looks like	How to keep it small
Egress	Per-GB charge every time data leaves a cloud or region; chatty cross-cloud calls add latency too	Keep services that talk to each other on one side; move data in bulk, rarely
Lowest common denominator	Avoiding Spanner, BigQuery, Cloud Run to stay "portable" — running a cloud as a VM host	Portability at the container/data boundary, not the service boundary
Duplicated tooling	Two IAM models, two monitoring stacks, two CI systems, two security reviews	One control plane where possible (GKE Enterprise); shared observability
Skills and on-call	Every engineer must know both platforms; incidents span two consoles	One primary cloud; a small named team owns the exception
Consistency	No cross-cloud transactions; identity federation and DNS stitched by hand	Draw the trust and data boundaries before the first deployment

THE RULE

Every cloud you add is a second set of everything — pay it knowingly, for a reason you can write down.

ACTIVITY A

The decision matrix — four workloads, six criteria

ACTIVITY A

1 · Order-export job

Runs nightly at 02:00 for ~15 min; reads the orders DB; writes a CSV to a bucket; nothing else runs then.

2 · Licensed reporting engine

Vendor binary, needs a kernel module and a static IP tied to its licence; used by 20 analysts 9–6.

3 · Public catalogue API

Bursty: 0 req/s at night, 400 req/s at 9 am; must answer in < 300 ms; stateless; developers only, no ops team.

4 · Bookshelf: web + api

Two small stateless Python services; web is public, api must be reachable by web only; team is this room.

Criterion	Card 1	Card 2	Card 3	Card 4
1 State	stateless — CR job ✓			
2 Traffic shape				
3 Latency budget				
4 Control				
5 Team				
6 Portability				

THE RULE

Copy the grid, tick a column per criterion, and let the ticks — not the loudest opinion — make the call.

Model answers — and the one criterion that decided each

ACTIVITY A • REVEAL

Card	Decision	The deciding criterion	Runner-up and why not
1 • Order-export job	Cloud Run job on Cloud Scheduler	Traffic shape: 15 min a day → pay for 15 min	VM: bills 24 h for 15 min of work
2 • Licensed reporting engine	Compute Engine VM (MIG of 1 for autohealing)	Control: kernel module + static licence IP	GKE: privileged pods possible but a cluster for one binary
3 • Public catalogue API	Cloud Run service, min-instances 1–2	Latency: no cold starts; Team: no ops team	GKE: fluency missing; VM: bursty traffic wastes capacity
4 • Bookshelf web + api	Two Cloud Run services; api internal + IAM-only	State: stateless; Team: this room	GKE: two services do not justify a cluster (yet)

Card 4 is the app you containerize and deploy in the next session — including how the api stays reachable by web only.

THE RULE

One criterion usually dominates — name it, and the other five become the checklist you run before you ship.

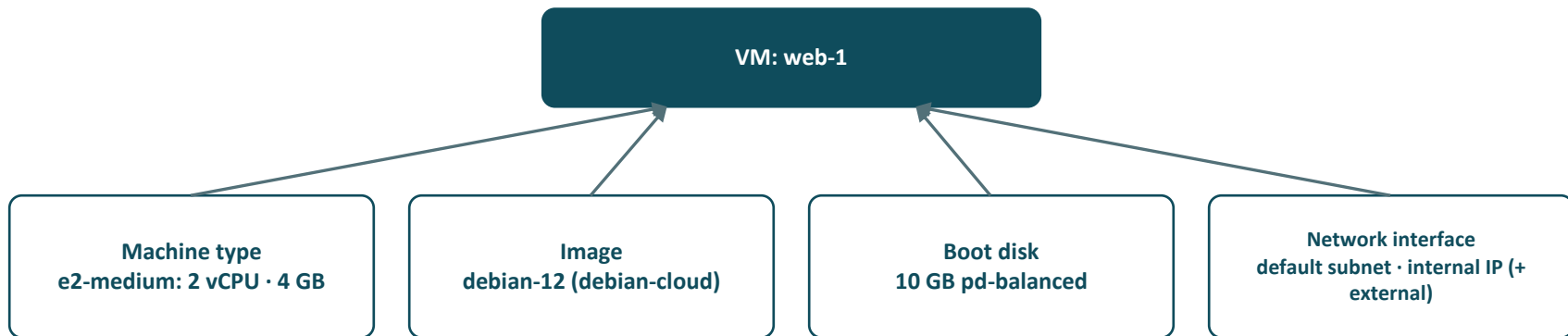
03

Compute Engine and instance groups

Virtual machines the Google way: machine families, images, disks — and turning one VM into a fleet that scales itself.

- 01 Landing in the sandbox
- 02 The compute spectrum, and choosing
- 03 Compute Engine and instance groups**
- 04 Google Kubernetes Engine
- 05 Cloud Functions and event-driven compute
- 06 Storage and databases
- 07 Networking: reaching and protecting the service

Anatomy of a VM: machine type, image, disk, network interface



Hanging off those four: metadata and startup scripts (what runs at boot), a service account (what the VM may call), network tags (which firewall rules apply), labels (billing). Regional vs zonal: a VM is always zonal; its boot disk lives in the same zone.

THE RULE

Machine type, image, disk, network — decide those four and the VM is decided.

Machine families — and how to read e2-standard-4

e2 - standard - 4

└─ 4 vCPUs
└─ memory ratio: standard 4 GB/vCPU · highmem 8 · highcpu 1-2
└─ series: E2 (family: general-purpose, cost-optimised)

Family	Series	Use it for	Note
General-purpose, cost-optimised	E2 (incl. shared-core e2-micro / small / medium)	Web servers, dev/test, small services — today's labs	Cheapest; CPU may be shared on micro/small
General-purpose, current gen	N4 · N4A (Arm/Axion) · N4D	Most production services	Hyperdisk only; better price/performance than N2
Performance	C4 · C4A (Arm/Axion) · C4D · C3	Latency-sensitive APIs, databases, CI runners	Highest all-core turbo; more expensive
Specialised	M (memory) · Z (storage) · A/G (GPU/accelerator)	In-memory DBs, big local SSD, ML training/inference	Quota needs requesting in advance

THE RULE

Default to E2 for anything small, N4 for anything real, and let a benchmark — not a blog post — move you to C4.

Image families: ask for debian-12, get today's patched build

WORKED EXAMPLE — IMAGE FAMILY RESOLVES TO A DATED BUILD

```
$ gcloud compute images describe-from-family debian-12 \
  --project=debian-cloud --format='value(name,creationTimestamp)'
debian-12-bookworm-v20260822 2026-08-22T09:14:03.201-07:00

$ gcloud compute images list --filter='family~^debian' \
  --format='table(name,family,project)' | head -4
NAME                                FAMILY    PROJECT
debian-11-bullseye-v20260822        debian-11  debian-cloud
debian-12-bookworm-v20260822        debian-12  debian-cloud
debian-13-trixie-v20260822          debian-13  debian-cloud
```

Three kinds of image

- Public — Google-maintained OS builds (debian-cloud, ubuntu-os-cloud, cos-cloud...)
- Custom — you bake your app in once; every VM from it boots ready
- Marketplace — vendor images (licensed software), billed per hour

Family = pointer to the newest image of a line. Templates should reference a family; audits should reference a dated image name.

THE RULE

Reference the family in templates, the dated image in audits — never a hard-coded image name in a script you will run next month.

Disks: Persistent Disk, Hyperdisk, Local SSD

	Persistent Disk	Hyperdisk	Local SSD
What it is	Network block storage (pd-standard / pd-balanced / pd-ssd)	Next-gen network block storage; IOPS/throughput set independently of size	NVMe flash on the host
Survives VM stop / delete	Yes (detachable, snapshot-able)	Yes (detachable, snapshot-able)	No — scratch only
Zonal / regional	Both (regional = 2-zone replica)	Zonal, some regional	Zonal, bound to the host
Works with	E2, N2, N2D, C3 ...	N4, C4, C4A, C3 ... (N4/C4: Hyperdisk only)	Series with local SSD slots
Typical use	Boot disks, general data	Databases, high-throughput apps	Caches, temp files, shuffle space

Backups: gcloud compute snapshots create — incremental, stored in Cloud Storage, restorable into any zone.

THE RULE

If the data must outlive the VM, it lives on a Persistent Disk or Hyperdisk — Local SSD is a cache, never a home.

COMPUTE ENGINE

Worked example: one VM from the CLI, and what the output says

WORKED EXAMPLE — CREATE A VM

```
$ gcloud compute instances create web-1 \  
  --machine-type=e2-micro \  
  --image-family=debian-12 --image-project=debian-cloud \  
  --boot-disk-size=10GB --boot-disk-type=pd-balanced \  
  --tags=http-server \  
  --metadata-from-file=startup-script=startup.sh  
Created [https://www.googleapis.com/compute/v1/projects/sandbox-rohan-07/  
zones/asia-south1-a/instances/web-1].  


| NAME  | ZONE          | MACHINE_TYPE | PREEMPTIBLE | INTERNAL_IP | EXTERNAL_IP   | STATUS  |
|-------|---------------|--------------|-------------|-------------|---------------|---------|
| web-1 | asia-south1-a | e2-micro     |             | 10.160.0.2  | 34.93.118.204 | RUNNING |


```

--tags=http-server
firewall rules target this tag, not the VM
name

--metadata-from-file=startup-script
runs as root at every boot — install,
configure, start

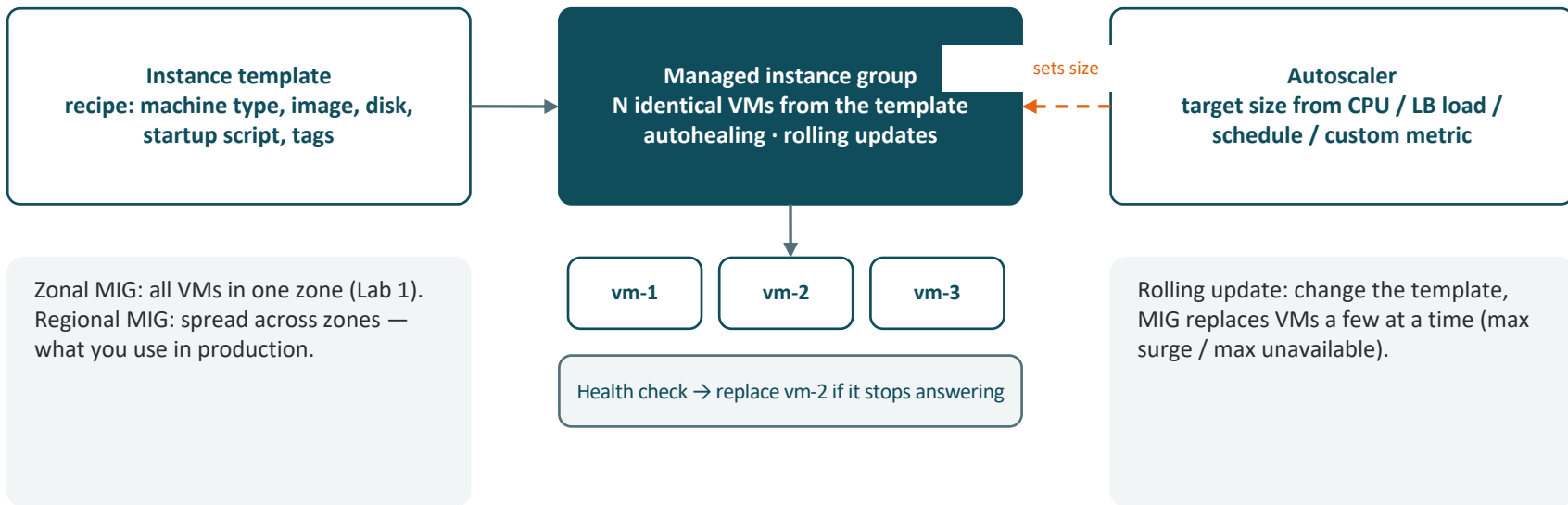
INTERNAL_IP always · EXTERNAL_IP only
because we did not pass --no-address

THE RULE

A VM is a command, not a wizard — put the command in a file and you already have your first template.

MANAGED INSTANCE GROUPS

From one VM to a fleet: template → group → autoscaler



THE RULE

Never hand-build the second VM — make a template, let the group stamp them out and keep them healthy.

AUTOSCALING

Worked example: CPU autoscaling, and why the cooldown matters

WORKED EXAMPLE — ATTACH AN AUTOSCALER

```
$ gcloud compute instance-groups managed set-autoscaling web-mig \
  --zone=asia-south1-a --min-num-replicas=1 --max-num-replicas=3 \
  --target-cpu-utilization=0.6 --cool-down-period=60
Created [https://www.googleapis.com/compute/v1/projects/
  sandbox-rohan-07/zones/asia-south1-a/autoscalers/web-mig-xkq2].
---
autoscalingPolicy:
  coolDownPeriodSec: 60
  cpuUtilization: {utilizationTarget: 0.6}
  maxNumReplicas: 3
  minNumReplicas: 1
  mode: ON
  name: web-mig-xkq2
  target: ../instanceGroupManagers/web-mig
```

The four knobs

- min / max — the floor and the ceiling; min ≥ 2 for anything you cannot afford to lose
- target — the utilisation you are steering toward, not a threshold
- cooldown — ignore a new VM while it boots, or you scale out twice
- scale-in is slow on purpose (10-min window): dips do not delete VMs

THE RULE

Autoscaling is a controller, not a switch: set the target you want, the bounds you can afford, and a cooldown as long as a boot.

A VM, then a fleet — create, template, group, autoscale, tear down

HANDS-ON · LAB 1

1

Unzip the kit and run steps 1–3 of lab1.sh as they are

startup.sh is in the kit: installs python3, serves one page on :80; the rule for tag http-server opens it

2

Open `http://EXTERNAL_IP` in a tab

You should see "Bookshelf baseline — Served by VM: bookshelf-vm" (give the startup script 60 s)

3

Fill TODO 1.4 — the instance template

Same recipe as the VM but e2-small; reference the image FAMILY, not a dated image

4

Fill TODO 1.5 — a zonal MIG of 2 from that template

--template, --size, --zone

5

Fill TODO 1.6 — autoscaling 1..3 on 60 % CPU, 60 s cooldown

Then list-instances; predict what the group size will be in 10 minutes

6

bash teardown.sh — before the break

Deletes the MIG, template, VM and firewall rule; safe to re-run

THE BLANKS

```
# TODO(lab-1.4)
gcloud compute instance-templates create
bookshelf-tmpl \
  --machine-type=_____ \
  --image-family=_____ --image-
project=_____ \
  --tags=http-server \
  --metadata-from-file=startup-
script=startup.sh

# TODO(lab-1.6)
--min-num-replicas=_____ --max-num-
replicas=_____ \
  --target-cpu-utilization=_____ --cool-
down-period=_____
```

Answers on the next slide. Console path: Compute Engine
→ VM instances → Create; Instance templates → Create;
Instance groups → Create (autoscaling section).

THE RULE

Template, group, autoscaler — three commands, and the second VM is never built by hand again.

What you should have seen — and the answers to the blanks

DEBRIEF · LAB 1

ANSWERS 1.4 · 1.5 · 1.6

```
--machine-type=e2-small
--image-family=debian-12 --image-project=debian-cloud
--template=bookshelf-tmpl --size=2 --zone=$ZONE
--min-num-replicas=1 --max-num-replicas=3
--target-cpu-utilization=0.6 --cool-down-period=60
```

PRODUCTION WOULD ADD

- Regional MIG (3 zones) · min 2 · a health check for autohealing
- A load balancer in front — chapter 7

EXPECTED — STEP 7, THEN AGAIN LATER

```
$ gcloud compute instance-groups managed list-instances \
    bookshelf-mig --zone=asia-south1-a
```

NAME	ZONE	STATUS	ACTION
INSTANCE_TEMPLATE			
bookshelf-mig-4k7p	asia-south1-a	RUNNING	NONE
bookshelf-mig-x2mc	asia-south1-a	RUNNING	NONE

~10 minutes later, VMs idle, min=1:

```
bookshelf-mig-4k7p asia-south1-a RUNNING NONE bookshelf-tmpl
```

COMMON STUMBLER

- Page not loading: firewall tag mismatch, or the startup script is still installing python3

THE RULE

The autoscaler did what you told it, not what you meant — bounds and targets are the specification.

04

Google Kubernetes Engine

Containers, why orchestration exists, what GKE gives you and when a cluster is warranted — with a cluster of your own.

- 01 Landing in the sandbox
- 02 The compute spectrum, and choosing
- 03 Compute Engine and instance groups
- 04 Google Kubernetes Engine**
- 05 Cloud Functions and event-driven compute
- 06 Storage and databases
- 07 Networking: reaching and protecting the service

Start your cluster now — it takes five to seven minutes

LAB 2 · KICK-OFF

RUN THIS NOW, THEN COME BACK TO THE SLIDES

```
gcloud services enable container.googleapis.com
gcloud container clusters create-auto hello-cluster --location asia-south1
# ... leave it running; open a second Cloud Shell tab (+) for the next slides
```

WHAT IT PRINTS WHEN DONE (5-7 MIN; VERSION AND IP DIFFER)

Note: The Kubelet readonly port (10255) is now deprecated. ...
Creating cluster hello-cluster in asia-south1... Cluster is being health-checked (Kubernetes Control Plane is healthy)...done.
Created [https://container.googleapis.com/v1/projects/sandbox-rohan-07/zones/asia-south1/clusters/hello-cluster].
kubeconfig entry generated for hello-cluster.

NAME	LOCATION	MASTER_VERSION	MASTER_IP	MACHINE_TYPE	NODE_VERSION	NUM_NODES	STATUS
hello-cluster	asia-south1	1.33.4-gke.1134000	34.93.201.17	e2-small	1.33.4-gke.1134000		RUNNING

Console path: Kubernetes Engine → Clusters → Create → Autopilot → hello-cluster, asia-south1 → Create.

One container is easy; fifty on twenty machines is scheduling

A container, in two lines

- Image = filesystem layers + a start command, stored in a registry
- Container = one process from that image, isolated by the kernel, up in milliseconds — no OS of its own

1

Scheduling

which machine runs which copy, given CPU/RAM

2

Self-healing

restart or replace a copy that dies; drain a bad node

3

Scaling

more copies on load, fewer when idle

4

Rollout

v1 → v2 gradually, roll back on failure

5

Discovery & routing

copies find each other; the outside finds the service

THE RULE

Every platform to the right of a VM answers these five questions — Kubernetes answers all five, and asks you to describe each answer in YAML.

What GKE gives you — and when a cluster is warranted

	Autopilot (recommended)	Standard
Nodes	Google-managed; you never see VMs	You size and manage node pools
Billing	Per pod CPU/RAM requested	Per node (VM price) + cluster fee
Upgrades, patching	Google, automatic	You schedule (or auto-upgrade channel)
Use when	Default for any new cluster — today's lab	GPUs/TPUs with custom drivers, DaemonSets, privileged pods, specific node config

Today's cluster is to see the shape — the six criteria from chapter 2 decide whether your service belongs in one.

A CLUSTER IS WARRANTED WHEN

- Many services share one platform
- Stateful / non-HTTP / custom networking needs
- The team already runs Kubernetes

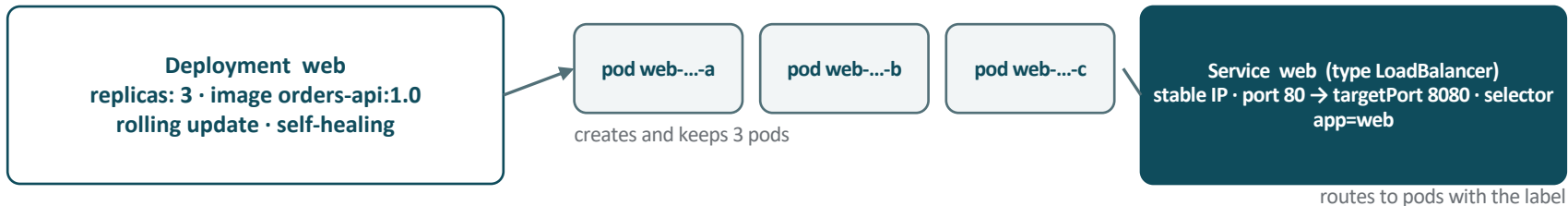
NOT WARRANTED WHEN

- One to three stateless HTTP services
- Nobody on the team has run a cluster
- "We might need it later"

THE RULE

GKE is the right answer to a platform question, and the wrong answer to a single-service question.

The four objects you touch today: Pod, Deployment, Service — and the autoscaler



TWO WAYS TO SAY THE SAME THING

```
# imperative — the first half of the lab
kubectl create deployment web --image=../orders-api:1.0
kubectl expose deployment web --type LoadBalancer --port 80 --target-port 8080
kubectl scale deployment web --replicas 3
# declarative — the same thing written down (second half)
kubectl apply -f deployment.yaml
```

Object	Means
Pod	the scheduled unit: containers sharing an IP
Deployment	N copies of a pod template, kept running, rolled out
Service	stable IP/name in front of matching pods
HPA	changes replicas from CPU/metrics

THE RULE

You describe the desired state — a Deployment of three, a Service in front — and the cluster's job is to make reality match it.

PREDICT THE CLUSTER

You delete one of the three pods by hand. Ten seconds later, how many pods?

`kubectl delete pod web-7c9f8-2kx9p` — the Deployment still says replicas: 3.

A) 2

You deleted one; it is gone.

B) 3 — a new one with a new name

The Deployment noticed the gap and filled it.

C) 3 — the same pod restarted
Kubernetes restores deleted pods.

D) 0 — deleting a pod deletes the Deployment
Pods own their Deployment.

A cluster, a deployment, a service — imperative, then declarative

LAB 2

1

Connect kubectl to your cluster (created at kick-off)

get-credentials, then `kubectl get nodes` — Autopilot may show none until a pod needs one

2

Create a Deployment from the sample image

`kubectl create deployment hello-web --image=us-docker.pkg.dev/google-samples/containers/gke/hello-app:2.0`

3

Expose it with a LoadBalancer Service, wait for the IP, open it

`kubectl expose ... --type LoadBalancer --port 80 --target-port 8080` · `kubectl get service hello-web -w`

4

Scale to 3, watch pods appear and nodes arrive

`kubectl scale deployment hello-web --replicas 3` · `kubectl get pods -o wide` · `kubectl get nodes`

5

Self-healing: delete a pod, watch it come back

`kubectl delete pod <one name>` · `kubectl get pods`

6

Declarative: read deployment.yaml, apply it, confirm nothing changed

`cat deployment.yaml` · `kubectl apply -f deployment.yaml` · `kubectl get deployment hello-web`

7

Teardown: delete the Service, then the cluster

`kubectl delete service hello-web` · `gcloud container clusters delete hello-cluster --location asia-south1 --quiet`

Console path

- Kubernetes Engine → Workloads → Deploy (image, name) → Expose (LoadBalancer, port 80)
- Workloads → hello-web → Actions → Scale
- Services & Ingress → hello-web → external IP
- Clusters → hello-cluster → Nodes · Delete

Commands on the next slide; reveal after that.

THE RULE

Deployment, Service, scale, self-heal, manifest — five kubectl verbs and you have seen the whole shape of a cluster.

Copy these, one block at a time

LAB 2 — CLOUD SHELL

```
# 1 connect
gcloud container clusters get-credentials hello-cluster --location asia-south1
kubectl get nodes
# 2 deployment · 3 service
kubectl create deployment hello-web --image=us-docker.pkg.dev/google-samples/containers/gke/hello-app:2.0
kubectl expose deployment hello-web --type LoadBalancer --port 80 --target-port 8080
kubectl get service hello-web -w          # Ctrl-C once EXTERNAL-IP is not <pending>
curl -s http://<EXTERNAL-IP>/
# 4 scale · 5 self-heal
kubectl scale deployment hello-web --replicas 3
kubectl get pods -o wide; kubectl get nodes
kubectl delete pod <pod-name>; sleep 5; kubectl get pods
# 6 declarative
unzip -q gcp-landscape-lab2-gke.zip && cat lab2/deployment.yaml
kubectl apply -f lab2/deployment.yaml; kubectl get deployment hello-web
# 7 teardown
kubectl delete service hello-web
gcloud container clusters delete hello-cluster --location asia-south1 --quiet
```

THE RULE

kubectl is the only new tool — every line describes desired state and asks the cluster to match it.

What you should have seen

LAB 2 · REVEAL

EXPECTED (IPS AND POD NAMES DIFFER)

```
$ kubectl expose deployment hello-web --type LoadBalancer --port 80 --target-port 8080
service/hello-web exposed
$ kubectl get service hello-web
NAME         TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
hello-web    LoadBalancer 34.118.224.9   34.93.156.203  80:31877/TCP     95s
$ curl -s http://34.93.156.203/
Hello, world!
Version: 2.0.0
Hostname: hello-web-6c9f8d7b4-2kx9p
$ kubectl scale deployment hello-web --replicas 3 && kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
hello-web-6c9f8d7b4-2kx9p           1/1     Running   0           4m
hello-web-6c9f8d7b4-7hrmz           1/1     Running   0           21s
hello-web-6c9f8d7b4-ml4wc           1/1     Running   0           21s
$ kubectl delete pod hello-web-6c9f8d7b4-2kx9p; sleep 5; kubectl get pods | wc -l
pod "hello-web-6c9f8d7b4-2kx9p" deleted
4                                     # header + 3 pods - one of them brand new
$ kubectl apply -f lab2/deployment.yaml
deployment.apps/hello-web configured
service/hello-web unchanged
```

COMMON STUMBLER

- EXTERNAL-IP <pending> for > 3 min: run get service -w again; the LB takes time
- get nodes empty: normal on Autopilot until pods are scheduled

TAKE WITH YOU

- desired state, not commands
- a Service is the stable address; pods are cattle
- the cluster bills whether or not it runs anything

THE RULE

Three pods, one address, a pod that came back by itself — and a cluster you must delete because it bills while idle.

05

Cloud Functions and event-driven compute

Functions that run in response to an HTTP call or an event, the trigger catalogue, and when a function beats a service.

- 01 Landing in the sandbox
- 02 The compute spectrum, and choosing
- 03 Compute Engine and instance groups
- 04 Google Kubernetes Engine
- 05 Cloud Functions and event-driven compute**
- 06 Storage and databases
- 07 Networking: reaching and protecting the service

CLOUD FUNCTIONS, NOW CLOUD RUN FUNCTIONS

A function is a handler plus a trigger — the platform owns the rest

WORKED EXAMPLE — AN HTTP FUNCTION (READ, NOT WRITTEN TODAY)

```
# main.py — an HTTP function
import functions_framework

@functions_framework.http
def hello(request):
    name = request.args.get("name", "world")
    return {"message": f"hello, {name}"}

# requirements.txt
functions-framework==3.10.2
```

A function IS a Cloud Run service: same revisions, IAM, logs. The --function form only removes the container and server work.

DEPLOY + CALL

```
$ gcloud run deploy hello --source . --function hello \
  --base-image python313 --region asia-south1 --allow-
unauthenticated
... Service URL: https://hello-482915730261.asia-south1.run.app
$ curl -s 'https://hello-482915730261.asia-
south1.run.app/?name=cloud'
{"message": "hello, cloud"}
```

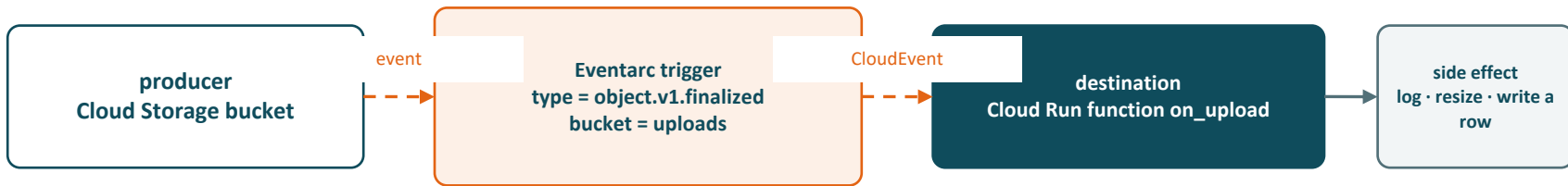
You write	The platform provides
one handler function	HTTPS server, container, base image patches
requirements.txt	build (buildpacks), scaling to zero and up
the trigger choice	delivery, retries, auth on the trigger

THE RULE

Write the handler, name the trigger, pick a base image — everything between the event and your code is the platform's.

EVENT-DRIVEN COMPUTE

Something happens, a message is produced, your code runs — nobody polls



WORKED EXAMPLE — THE HANDLER SIDE OF THE CONTRACT (READ, NOT WRITTEN TODAY)

```
@functions_framework.cloud_event
def on_upload(cloud_event):
    data = cloud_event.data                # the payload: bucket, name, size,
    contentType ...
    print(f"new object: gs://{data['bucket']}/{data['name']}")
    # envelope: cloud_event['id'], ['type'] =
    google.cloud.storage.object.v1.finalized, ['source'], ['time']
```

Request-driven: someone waits for the reply. Event-driven: something already happened; your job is to react — possibly twice.

Contract	Means for your handler
at-least-once delivery	may run twice for one event → be idempotent
retries on error	a raised exception = retry with backoff
no caller waiting	return nothing useful; the work is the side effect
envelope + data	type/source/id + the producer's payload

THE RULE

Design an event handler for the message arriving twice and nobody waiting — that is the whole difference from an HTTP endpoint.

The trigger catalogue — and the three IAM grants that make a trigger work

Source	Event type (examples)	Typical use
Cloud Storage	google.cloud.storage.object.v1.finalized / deleted	process an upload, resize, index — Lab 3
Pub/Sub	google.cloud.pubsub.topic.v1.messagePublished	decouple producers and consumers; fan-out
Cloud Audit Logs	google.cloud.audit.log.v1.written (+ service/method filter)	react to admin actions: VM created, IAM changed
Firestore	google.cloud.firestore.document.v1.written	on-write side effects for app data
Cloud Scheduler	cron → Pub/Sub message	the nightly job from chapter 2

THE THREE GRANTS (FULL FORM ON THE LAB SLIDE)

```
# 1 Storage service agent → publish
SA=$(gcloud storage service-agent)
... add-iam-policy-binding $P \
  --member=serviceAccount:$SA \
  --role=roles/pubsub.publisher
# 2 + 3 trigger identity → receive, invoke
TSA=$PN-compute@developer.gserviceaccount.com
... --member=serviceAccount:$TSA \
  --role=roles/eventarc.eventReceiver
... --member=serviceAccount:$TSA \
  --role=roles/run.invoker
```

THE TRIGGER ITSELF

```
$ gcloud eventarc triggers create on-upload --location=asia-south1 --destination-run-service=gcs-function \
  --event-filters="type=google.cloud.storage.object.v1.finalized" --event-filters="bucket=uploads-..." --service-account=$TSA
```

THE RULE

An event trigger is three permissions and one filter — when a function "never fires", it is almost always one of the three.

The same criteria as chapter 2, applied to the right edge

	Cloud Run function	Cloud Run service	Cloud Run job
Shape	one handler, one trigger	an app with routes; container optional	run to completion, no endpoint
Starts from	HTTP call or an event	HTTP/gRPC request (or an event via Eventarc)	a schedule, a command, a workflow
Runs for	seconds (events default 60 s; HTTP up to 60 min)	per request, up to 60 min	up to 24 h per task; parallel tasks; retries
Best for	glue, webhooks, on-upload / on-message reactions	APIs, web apps — the next session	batch, migrations, reports
Anti-pattern	a 40-minute batch in a function	a REST API split into ten functions	a job that polls for HTTP requests

GKE enters when many of these share a platform and the team runs Kubernetes — chapter 4's criteria, not the shape of one workload.

THE RULE

Trigger + short handler → function; an app people call → service; work that ends → job — and none of them needs a cluster.

An HTTP function, then a function that reacts to an upload

LAB 3

1

Unzip; read both functions (no edits)

```
cat hello-function/main.py gcs-function/main.py
```

2

Deploy the HTTP function and call it

```
gcloud run deploy hello-function --source . --function hello --base-image python313 ... · curl ?name=bookshelf
```

3

Create the upload bucket

```
gcloud storage buckets create gs://bookshelf-uploads-<project> --location=asia-south1 --uniform-bucket-level-access
```

4

The three grants (Storage service agent → pubsub.publisher; trigger SA → eventReceiver + run.invoker)

Commands on the next slide

5

Deploy the event function, then create the Eventarc trigger on object finalized

```
gcloud run deploy gcs-function --function on_upload ... · gcloud eventarc triggers create on-upload ...
```

6

Upload a file, read the log line

```
echo hello > note.txt; gcloud storage cp note.txt gs://... · gcloud run services logs read gcs-function --limit 5
```

7

Teardown (or leave the bucket for the close)

```
delete the trigger, both functions, the bucket
```

Console path

- Cloud Run → Write a function → runtime Python 3.13 → paste main.py → entry point
- Cloud Run → gcs-function → Triggers → Add Eventarc trigger → Cloud Storage, object finalized, bucket
- IAM & Admin → grant the three roles
- Cloud Storage → bucket → Upload files · Cloud Run → Logs

Commands on the next slide; reveal after that.

THE RULE

One handler per trigger, three grants per event source — and a file landing in a bucket ran your code without a server anywhere.

LAB 3 • COMMANDS

Copy these, one block at a time

LAB 3 — CLOUD SHELL

```
PROJECT=$(gcloud config get-value project); PN=$(gcloud projects describe $PROJECT --format='value(projectNumber)'); R=asia-south1
# 2 HTTP function
(cd hello-function && gcloud run deploy hello-function --source . --function hello --base-image python313 --region $R --allow-unauthenticated)
curl -s "$(gcloud run services describe hello-function --region $R --format='value(status.url)')/?name=bookshelf"; echo
# 3 bucket
gcloud storage buckets create gs://bookshelf-uploads-$PROJECT --location=$R --uniform-bucket-level-access
# 4 the three grants
gcloud projects add-iam-policy-binding $PROJECT --member=serviceAccount:$(gcloud storage service-agent) --role=roles/pubsub.publisher
gcloud projects add-iam-policy-binding $PROJECT --member=serviceAccount:$PN-compute@developer.gserviceaccount.com --role=roles/eventarc.eventReceiver
gcloud projects add-iam-policy-binding $PROJECT --member=serviceAccount:$PN-compute@developer.gserviceaccount.com --role=roles/run.invoker
# 5 event function + trigger
(cd gcs-function && gcloud run deploy gcs-function --source . --function on_upload --base-image python313 --region $R --no-allow-unauthenticated)
gcloud eventarc triggers create on-upload --location=$R --destination-run-service=gcs-function --destination-run-region=$R \
  --event-filters="type=google.cloud.storage.object.v1.finalized" --event-filters="bucket=bookshelf-uploads-$PROJECT" \
  --service-account=$PN-compute@developer.gserviceaccount.com
# 6 upload, read the log (wait ~30 s after the first upload)
echo hello > note.txt; gcloud storage cp note.txt gs://bookshelf-uploads-$PROJECT/; sleep 30
gcloud run services logs read gcs-function --region $R --limit 5
# 7 teardown
gcloud eventarc triggers delete on-upload --location=$R --quiet; gcloud run services delete gcs-function --region $R --quiet
gcloud run services delete hello-function --region $R --quiet; gcloud storage rm -r gs://bookshelf-uploads-$PROJECT
```

THE RULE

Deploy, grant, trigger, upload — the function's code never changed; the wiring is the work.

What you should have seen

LAB 3 · REVEAL

EXPECTED

```
$ curl -s "${...}/?name=bookshelf"
{"from":"cloud-run-function","message":"hello, bookshelf"}
$ gcloud eventarc triggers create on-upload ...
Creating trigger [on-upload] in project [sandbox-rohan-07], location [asia-south1]...done.
WARNING: It may take up to 2 minutes for the new trigger to become active.
$ gcloud storage cp note.txt gs://bookshelf-uploads-sandbox-rohan-07/
$ gcloud run services logs read gcs-function --region asia-south1 --limit 5
2026-09-07 11:02:44  new object: gs://bookshelf-uploads-sandbox-rohan-07/note.txt  size=6 bytes
type=text/plain
2026-09-07 11:02:44  event id=12345678901234 type=google.cloud.storage.object.v1.finalized
2026-09-07 11:02:44  POST 200 41ms /
```

EXTENSION

- Upload the same file again and compare the two event ids — same object, two events, two runs.

NO LOG LINE?

- gcloud eventarc triggers describe on-upload — an error names the missing grant
- first trigger takes up to 2 min: upload again

TAKE WITH YOU

- three grants, one filter
- the handler may run twice
- a function is a Cloud Run service

THE RULE

A file landed, a message travelled, your handler ran and logged it — event-driven compute in one upload.

06

Storage and databases

Buckets, classes, lifecycle rules, disks, file shares and the database portfolio — with Postgres and Firestore at the keyboard.

- 01 Landing in the sandbox
- 02 The compute spectrum, and choosing
- 03 Compute Engine and instance groups
- 04 Google Kubernetes Engine
- 05 Cloud Functions and event-driven compute
- 06 Storage and databases**
- 07 Networking: reaching and protecting the service

Start your Cloud SQL instance now — it takes five to ten minutes

LAB 5 · KICK-OFF

RUN THIS NOW, THEN COME BACK TO THE SLIDES

```
gcloud services enable sqladmin.googleapis.com firestore.googleapis.com
gcloud sql instances create bookshelf-pg --database-version=POSTGRES_17 --tier=db-f1-micro \
  --region=asia-south1 --edition=ENTERPRISE --root-password='Bookshelf#2026'
# ... leave it running; use a second Cloud Shell tab for the next slides
```

WHAT IT PRINTS WHEN DONE (5-10 MIN; ADDRESS DIFFERS)

```
Creating Cloud SQL instance for POSTGRES_17...done.
Created [https://sqladmin.googleapis.com/sql/v1beta4/projects/sandbox-rohan-07/instances/bookshelf-pg].
```

NAME	DATABASE_VERSION	LOCATION	TIER	PRIMARY_ADDRESS	PRIVATE_ADDRESS	STATUS
bookshelf-pg	POSTGRES_17	asia-south1-b	db-f1-micro	34.93.77.140	-	RUNNABLE

Console path: SQL → Create instance → PostgreSQL → bookshelf-pg · asia-south1 · Enterprise · Sandbox (shared core) preset → Create.

Buckets and objects: a flat, globally named, 11-nines object store

bucket: docs-sandbox-rohan-07 (region asia-south1)

exports/catalogue-1.txt

exports/catalogue-2.txt

images/cover-42.jpg

backups/2026-09-01.sql.gz

objects: key → bytes + metadata (content-type, storage class, generation, custom keys). "exports/" is part of the key, not a folder.

Location type	Where	Use
Region	one region (asia-south1)	app data next to compute; cheapest
Dual-region	two named regions	active-active, DR
Multi-region	a continent (asia)	serving to a wide audience

Access

- uniform bucket-level access: IAM only — always on
- signed URL: time-limited link to one object, no account needed
- public buckets: a deliberate, rare, audited choice

THE RULE

A bucket is a key-value store with a globally unique name — plan the name, the location and IAM before the first upload.

CLOUD STORAGE

Storage classes: the same bucket, priced by how often you read

Class	Minimum storage	Retrieval fee	Designed for	Availability SLA (region)
Standard	none	none	hot data, serving, short-lived	99.9 %
Nearline	30 days	yes	read about once a month: backups, recent exports	99.0 %
Coldline	90 days	yes	read about once a quarter	99.0 %
Archive	365 days	yes	read less than once a year: DR, compliance	99.0 %
Rapid	none	none	zonal, I/O-intensive AI/ML and analytics scratch	99.9 % (zone)

WORKED EXAMPLE — CREATE, THEN HAND CLASS DECISIONS TO AUTOCLASS

```
$ gcloud storage buckets create gs://docs-sandbox-rohan-07 --location=asia-south1 \  
  --default-storage-class=STANDARD --uniform-bucket-level-access \  
Creating gs://docs-sandbox-rohan-07/...  
$ gcloud storage buckets update gs://docs-sandbox-rohan-07 --enable-autoclass # let the bucket decide
```

THE RULE

Colder classes are cheaper to keep and dearer to touch — pick by read frequency, or turn on Autoclass and let the data decide.

Lifecycle rules: the bucket cleans up after itself

WORKED EXAMPLE — LIFECYCLE.JSON AND HOW IT IS APPLIED

```
{ "rule": [
  { "action": { "type": "SetStorageClass", "storageClass": "NEARLINE" },
    "condition": { "age": 30, "matchesStorageClass": ["STANDARD"] } },
  { "action": { "type": "Delete" },
    "condition": { "numNewerVersions": 3, "isLive": false } },
  { "action": { "type": "Delete" },
    "condition": { "age": 365 } }
] }
```

```
$ gcloud storage buckets update gs://docs-sandbox-rohan-07 --lifecycle-
file=lifecycle.json
Updating gs://docs-sandbox-rohan-07/...
```

Condition	Means
age	days since the object was created
matchesStorageClass	only objects currently in these classes
numNewerVersions	at least N newer versions exist
isLive	true = current version; false = old versions
matchesPrefix	key starts with... (e.g. exports/)

Rules evaluate about once a day; a transition is asynchronous. Actions cannot be undone — test on a scratch bucket first.

THE RULE

Write the retention policy of your data as rules on the bucket — not as a cron job someone forgets.

VERSIONING AND RETENTION

Versioning protects you from yourself; retention protects data from everyone

	Object versioning	Retention policy	Object hold
Protects against	accidental overwrite / delete	any delete or overwrite before the object is N old	delete of one object while the hold is set
Scope	bucket (all objects)	bucket (all objects)	one object
Reversible?	yes — turn off, delete old versions	yes until LOCKED; a locked policy is permanent	yes — release the hold
Cost	storage per kept version	none, but nothing can be deleted early	none
Command	buckets update --versioning	buckets update --retention-period=P1D	objects update --temporary-hold

DO NOT

- Lock a retention policy on anything but a compliance bucket — a locked policy cannot be shortened, and the bucket cannot be deleted until every object

THE RULE

Versioning is undo; retention is a promise — make the promise only where a regulator would ask for it.

PERSISTENT DISKS AND SHARED FILE STORAGE

Block, file, object: three shapes of storage for three kinds of access

Block

Persistent Disk · Hyperdisk

attached to one VM; OS formats it

boot disks, databases, anything needing a filesystem on one machine

```
gcloud compute disks create
```

File

Filestore (NFS)

mounted by many VMs / pods

shared uploads, home dirs, legacy apps needing a POSIX path

```
gcloud filestore instances create
```

Object

Cloud Storage

HTTP API from anywhere; no mount

files a service reads/writes whole; backups; static assets

```
gcloud storage cp
```

Cloud Run instances have only ephemeral memory-backed disk — they mount a bucket (Cloud Storage volumes) or an NFS share when they need files.

THE RULE

If more than one instance must see the file, it is not a disk — it is a bucket, or a share if it must be a path.

Cloud SQL, Spanner, Firestore, BigQuery — four shapes, not four sizes

Cloud SQL

Relational, one region

MySQL · PostgreSQL · SQL Server;
vertical scale, read replicas; the
default OLTP choice

e.g. orders, users, inventory

Spanner

Relational, horizontal, global

SQL with strong consistency across
regions; scales by adding nodes;
higher price floor

e.g. ledgers, global inventory, > 1
DB machine

Firestore

Document, real-time

Flexible JSON-like documents, live
listeners, mobile/offline SDKs,
serverless

e.g. user profiles, sessions, app
state

BigQuery

Analytical warehouse

Columnar, SQL over petabytes, pay
per byte scanned; not for per-
request lookups

e.g. dashboards, log analysis, ML
features

Also on the shelf: AlloyDB (high-end PostgreSQL), Bigtable (wide-column, huge write throughput), Memorystore (Redis cache).

THE RULE

Transactions for one region → Cloud SQL; transactions everywhere → Spanner; documents → Firestore; questions over history → BigQuery.

Managed PostgreSQL: what Google runs, what you still decide

Decision	Options	Lab 5 / production
Edition and machine	Enterprise · Enterprise Plus; shared-core (db-f1-micro) or dedicated vCPU/RAM	db-f1-micro, Enterprise / dedicated cores, sized on metrics
Availability	single zone · regional HA (standby + automatic failover)	single zone / regional HA for anything customer-facing
Backups	automated daily + point-in-time recovery; on-demand backups	one on-demand backup in the lab / automated + PITR always
Connectivity	public IP + authorized networks · private IP in the VPC · Auth Proxy / connectors (IAM + TLS)	public IP + gcloud sql connect / private IP + Auth Proxy
Access	database users + passwords · IAM database authentication	postgres user in the lab / IAM auth for services

```
$ gcloud sql connect bookshelf-pg --user=postgres --database=bookshelf
Allowlisting your IP for incoming connection for 5 minutes...done.
bookshelf=> SELECT title, year FROM books ORDER BY year;
```

THE RULE

Google runs the database engine; you still own the shape, the availability, the backups and — above all — how it is reached.

FIRESTORE

A document database: collections, documents, fields — no schema to migrate

THE DATA MODEL — A TREE OF COLLECTIONS AND DOCUMENTS

```
projects/sandbox-rohan-07/databases/(default)/documents
└─ books/                                collection
    └─ book-1                            document
        title: "Release It!"  author: "Nygard"  year: 2018  tags: ["ops",
"resilience"]
        └─ reviews/                    subcollection
            └─ r-1 { stars: 5, text: "..."}
└─ book-2                                document (different fields are fine)
    title: "SRE"  year: 2016  free: true
```

Trait	Means for you
no schema	validate in the app / security rules; version your documents
indexes	single-field automatic; composite on demand (the error links to it)
consistency	strong reads; transactions across documents
real-time	listeners push changes to clients
pricing	per read/write/delete + storage — avoid chatty polling

Lab 5: create the database, add documents in the console, read and query them with the REST API from Cloud Shell — the same calls the client libraries make.

THE RULE

Model documents the way the app reads them — a document per screen, not a table per entity — and let the index errors tell you what to add.

MEMORystore

An in-memory cache in your VPC: Redis or Valkey, milliseconds, no disk



WORKED EXAMPLE — CREATE, THEN CACHE-ASIDE BY HAND

```
$ gcloud redis instances create bookshelf-cache --size=1 --region=asia-south1 --  
network=default # ~5 min; private IP only  
# from a VM in the same VPC:  
$ redis-cli -h 10.166.0.3 SET book:1 'Release It!' EX 300 → OK  
$ redis-cli -h 10.166.0.3 GET book:1 → "Release It!"
```

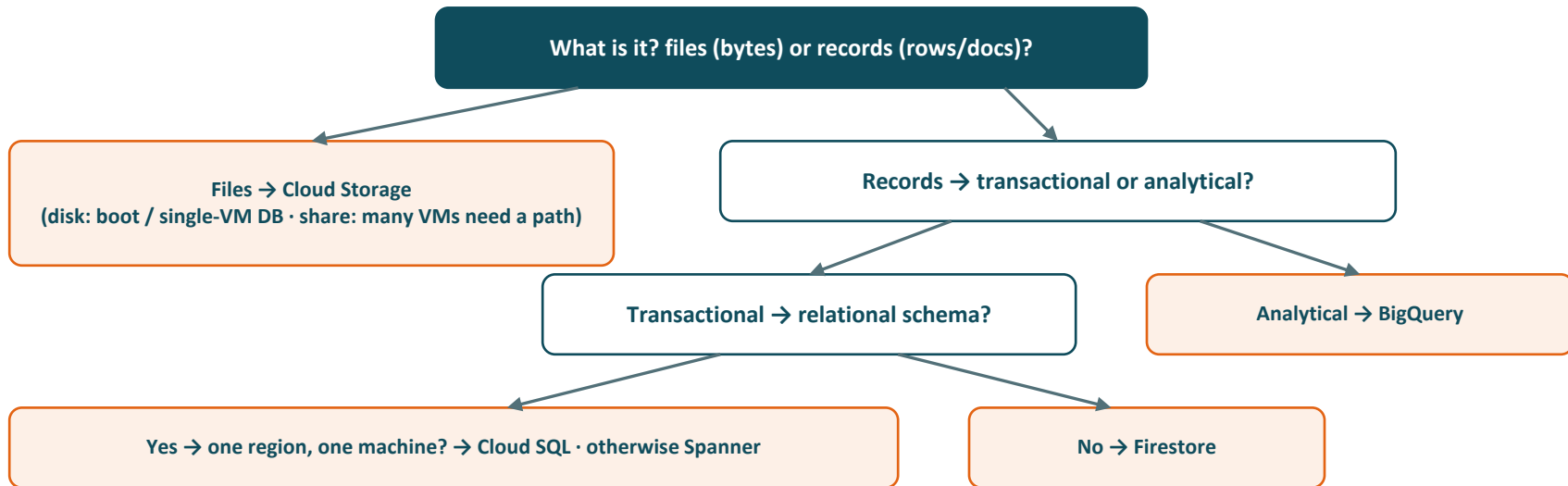
Use it for	Not for
hot reads in front of Cloud SQL	the system of record
sessions, counters, rate limits	large objects (a bucket)
leaderboards, queues, pub/sub	data you cannot rebuild

Tiers: Basic (single node, no replication — dev) · Standard (replica + automatic failover). It is a cache: design every read to survive a miss.

THE RULE

A cache buys milliseconds and takes load off the database — but only the database is the truth, and the VPC is the only door.

One decision tree for a given workload



Overlays: Memystore (Redis) in front of hot reads · a lifecycle rule on every bucket · backups scheduled on every database.

THE RULE

Files, records, transactional, relational, one-region — five yes/no questions place any workload on the shelf.

ACTIVITY B

Pick the store — five workloads, one service each

ACTIVITY B

#	Workload	Your pick	Why (one line)
1	Product images uploaded by sellers, served on every page view, kept forever		
2	Orders and payments for one country; strict consistency; 2 000 writes/s at peak		
3	"Revenue by category by hour, last 3 years" for the finance dashboard		
4	Per-user shopping carts with live updates on web and mobile		
5	Bookshelf's catalogue — 4 books today, maybe 4 000 next year; read by the api on every request		

Copy the table; fill the last two columns. Answers on the next slide.

THE RULE

Run the five questions, write one line of reasoning — the line is what survives the code review.

Model answers

ACTIVITY B · REVEAL

#	Workload	Pick	Why
1	Seller product images, served every page view	Cloud Storage, Standard (multi-region if global) + Cloud CDN	Files; hot; served over HTTP — never on a disk
2	Orders and payments, one country, strict consistency	Cloud SQL (PostgreSQL)	Relational, transactional, one region; Spanner only for multi-region writes
3	Revenue by category by hour, 3 years	BigQuery	Analytical scan over history; not per-request
4	Per-user carts, live on web and mobile	Firestore	Document per user; real-time listeners; offline SDKs
5	Bookshelf catalogue read by the api	Cloud SQL (+ Memorystore later)	Relational and tiny; api stays stateless; today's in-memory list is the placeholder

THE RULE

If two people picked differently and both can say why, the workload description was missing a constraint — write it down.

Postgres from psql, then Firestore documents

LAB 5

1

Cloud SQL — create the bookshelf database, connect

```
gcloud sql databases create · gcloud sql connect bookshelf-pg --user=postgres --database=bookshelf
```

2

In psql: create the books table, insert four rows, query, quit

```
CREATE TABLE · INSERT · SELECT ... ORDER BY year · \q (SQL on the next slide)
```

3

An on-demand backup, and where the instance lives

```
gcloud sql backups create · gcloud sql instances describe (public IP, zone, tier)
```

4

Firestore — create the database, add two documents in the console

```
gcloud firestore databases create · Firestore → Data → Start collection: books → book-1 {title, author, year}, book-2
```

5

Read and query the documents from Cloud Shell (REST, the same calls the SDKs make)

```
GET .../documents/books · POST ...:runQuery with year >= 2017
```

6

Teardown

```
gcloud sql instances delete bookshelf-pg · gcloud firestore databases delete --database='(default)'
```

Console path

- SQL → bookshelf-pg → Databases → Create; Cloud SQL Studio for a browser psql
- SQL → bookshelf-pg → Backups → Create backup
- Firestore → Create database → Native, asia-south1
- Firestore → Data → Start collection → add document

Commands on the next slide; reveal after that.

THE RULE

A relational table you wrote SQL against and a document you never gave a schema — the two shapes, both managed, both reached from a shell.

LAB 5 · COMMANDS

Copy these, one block at a time

LAB 5 — CLOUD SHELL AND PSQL

```
PROJECT=$(gcloud config get-value project)
# 1 database + connect (password: Bookshelf#2026)
gcloud sql databases create bookshelf --instance=bookshelf-pg
gcloud sql connect bookshelf-pg --user=postgres --database=bookshelf
# 2 inside psql
CREATE TABLE books (id serial PRIMARY KEY, title text NOT NULL, author text, year int);
INSERT INTO books (title, author, year) VALUES ('The Pragmatic Programmer','Hunt & Thomas',1999),
('Designing Data-Intensive Applications','Kleppmann',2017),('Site Reliability Engineering','Beyer et al.',2016),('Release It!','Nygard',2018);
SELECT title, year FROM books WHERE year >= 2017 ORDER BY year;      \q
# 3 backup + describe
gcloud sql backups create --instance=bookshelf-pg
gcloud sql instances describe bookshelf-pg --format='value(ipAddresses[0].ipAddress,gceZone,settings.tier)'
# 4 Firestore database — then add book-1 and book-2 in the console (Firestore → Data → Start collection: books)
gcloud firestore databases create --location=asia-south1 --type=firestore-native
# 5 read + query via REST
TOKEN=$(gcloud auth print-access-token); FS="https://firestore.googleapis.com/v1/projects/$PROJECT/databases/(default)/documents"
curl -s -H "Authorization: Bearer $TOKEN" "$FS/books" | head -30
curl -s -X POST -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' "$FS:runQuery" \
-d
'{"structuredQuery":{"from":[{"collectionId":"books"}],"where":{"fieldFilter":{"field":{"fieldPath":"year"},"op":"GREATER_THAN_OR_EQUAL","value":{"integerValue":"2017"}}}}}'
# 6 teardown
gcloud sql instances delete bookshelf-pg --quiet; gcloud firestore databases delete --database='(default)' --quiet
```

THE RULE

SQL for the table, JSON for the document — and gcloud, psql and curl were the only tools.

What you should have seen

LAB 5 · REVEAL

EXPECTED

```
$ gcloud sql connect bookshelf-pg --user=postgres --database=bookshelf
Allowlisting your IP for incoming connection for 5 minutes...done.
Password:
psql (17.x) ... SSL connection
bookshelf=> SELECT title, year FROM books WHERE year >= 2017 ORDER BY year;
      title                | year
-----+-----
 Designing Data-Intensive Applications | 2017
 Release It!                 | 2018
(2 rows)
$ gcloud sql backups create --instance=bookshelf-pg
Backing up Cloud SQL instance...done.
$ curl ... "$FS:runQuery" -d '{...year >= 2017...}'
[{"document":{"name":"projects/sandbox-rohan-07/databases/(default)/documents/books/book-1",
  "fields":{"title":{"stringValue":"Release
It!"},"author":{"stringValue":"Nygard"},"year":{"integerValue":"2018"}}, ...}]
```

COMMON STUMBLES

- connect times out → the instance is still creating; gcloud sql instances list
- relation already exists → you re-ran CREATE; carry on
- runQuery 400 → check the quotes around \$FS

TAKE WITH YOU

- Cloud SQL: shape + backups + how it is reached
- Firestore: documents, no schema, indexes on demand

THE RULE

Two rows from SQL, one document from a query — and a five-minute allowlist that told you how not to do it in production.

07

Networking: reaching and protecting the service

VPCs, subnets, firewall rules, routes, load balancers, private access and DNS: who can reach what, and how.

- 01 Landing in the sandbox
- 02 The compute spectrum, and choosing
- 03 Compute Engine and instance groups
- 04 Google Kubernetes Engine
- 05 Cloud Functions and event-driven compute
- 06 Storage and databases
- 07 Networking: reaching and protecting the service**

VPC NETWORKS

A VPC is global; its subnets are regional; its IPs are yours to plan

VPC: prod-vpc (global) — routes · firewall rules · peering

subnet asia-south1
10.10.0.0/24

subnet europe-west1
10.20.0.0/24

subnet us-central1
10.30.0.0/24

vm 10.10.0.2

vm 10.10.0.3

vm 10.20.0.2

vm 10.30.0.2

VMs in different regions talk over internal IPs — one network, no VPN, no peering.

Auto mode

a subnet in every region from 10.128.0.0/9; the 'default' network

Custom mode

no subnets until you create them; you choose every range — production

THE RULE

One network per environment, one subnet per region you actually use, and ranges you chose on purpose.

IP ADDRESS PLANNING

CIDR, reserved addresses, and ranges that will not collide later

WORKED EXAMPLE — HOW MANY VMS FIT IN A /24

```
10.10.0.0/24 → 256 addresses
- 10.10.0.0    network address
- 10.10.0.1    default gateway
- 10.10.0.254  reserved (second-to-last)
- 10.10.0.255  broadcast
= 252 usable VM addresses

/22 = 1 024 (1 020 usable)   /20 = 4 096 (4 092 usable)
```

External IPs are a separate pool: ephemeral by default; reserve a static one only when DNS or a licence needs it (Lab 4 uses none).

Plan	Range	Purpose
prod	10.10.0.0/16	environment
prod / asia-south1 / apps	10.10.0.0/22	1 020 VMs
prod / asia-south1 / data	10.10.4.0/24	databases
gke pods (secondary)	10.11.0.0/17	32 k pods
staging	10.20.0.0/16	never overlaps prod

THE MISTAKE THAT CANNOT BE FIXED

- Two networks with overlapping ranges can never be peered or VPN-connected — renumbering means rebuilding

THE RULE

Write the address plan before the first subnet — ranges are the one thing in cloud you cannot rename later.

FIREWALL RULES

Stateful, network-scoped, matched by priority — and targeted by tag or identity

WORKED EXAMPLE — SSH ONLY FROM GOOGLE'S IAP RANGE

```
$ gcloud compute firewall-rules create allow-iap-ssh \
  --network=prod-vpc --direction=INGRESS --action=ALLOW \
  --rules=tcp:22 --source-ranges=35.235.240.0/20 --target-tags=private-vm
Creating firewall...done.
NAME          NETWORK  DIRECTION  PRIORITY  ALLOW  DENY  DISABLED
allow-iap-ssh  prod-vpc  INGRESS    1000      tcp:22  False
```

Implied rule (every VPC)	Priority	Effect
deny all ingress	65535	nothing reaches a VM unless a rule allows it
allow all egress	65535	VMs may reach anything (routes permitting)

Field	Options	Lab 4 value
direction	INGRESS · EGRESS	INGRESS
action / priority	ALLOW · DENY / 0–65535 (lower wins)	ALLOW / 1000
source	CIDRs · source tags · source service accounts	35.235.240.0/20 (IAP)
target	all · --target-tags · --target-service-accounts	tag private-vm
rules	protocol:ports	tcp:22

THE RULE

Default-deny in, default-allow out — every open port is a rule you wrote, so make its source as narrow as the job allows.

ROUTES

Routes decide where a packet goes next — the default one points at the internet

WORKED EXAMPLE — WHAT A FRESH CUSTOM VPC WITH ONE SUBNET HAS

```
$ gcloud compute routes list --filter="network:prod-vpc"
```

NAME	NETWORK	DEST_RANGE	NEXT_HOP	PRIORITY
default-route-3f1a9c0b2e7d4a61	prod-vpc	10.10.0.0/24	prod-vpc	0
default-route-9b8e2d4c6a1f0e37	prod-vpc	0.0.0.0/0	default-internet-gateway	1000



Need	Fix
Internet from private VMs (apt, pip, webhooks)	Cloud NAT on the subnet — managed, no VM to run
Google APIs from private VMs (storage, run, ...)	Private Google Access on the subnet — next slides

THE RULE

No external IP means no internet by default — add Cloud NAT for the world, Private Google Access for Google, and nothing else.

LOAD BALANCING

The load-balancer family: Application vs Network, external vs internal, global vs regional

	External (internet-facing)	Internal (from your VPC)
Application LB (L7 · HTTP/S)	Global (anycast IP, multi-region backends, Cloud CDN, Cloud Armor) · Regional · Classic	Regional · Cross-region
Proxy Network LB (L4 · TCP/SSL proxy)	Global · Regional · Classic	Regional · Cross-region
Passthrough Network LB (L4 · TCP/UDP, source IP kept)	Regional (Global in preview)	Regional

CHOOSE THE APPLICATION LB WHEN

- HTTP/S, URL- or host-based routing, TLS at the edge, CDN, WAF (Cloud Armor), Cloud Run or MIG backends

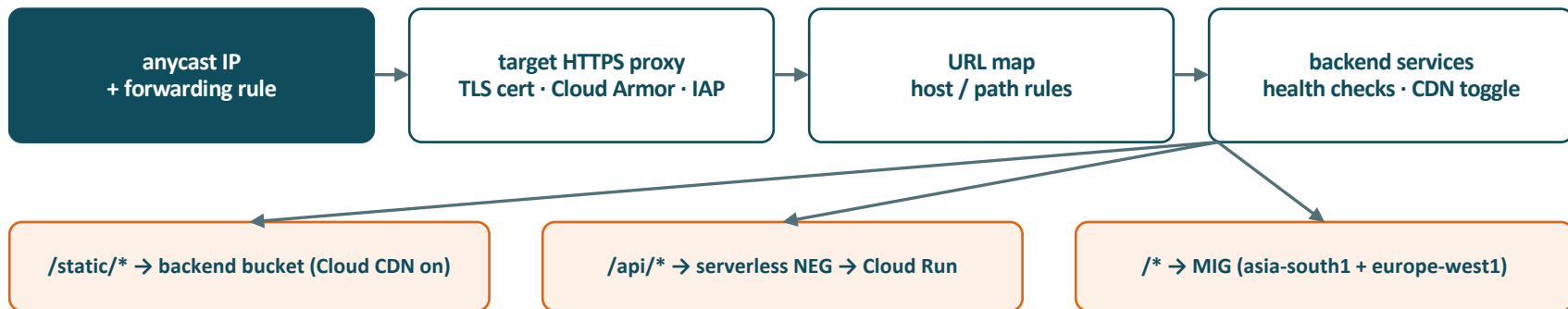
CHOOSE A NETWORK LB WHEN

- Non-HTTP protocols, UDP, TLS passthrough to the backend, the client's source IP must be preserved

THE RULE

For an HTTP service the answer is almost always the global external Application Load Balancer — everything else is an exception you can name.

Anatomy of a global external Application Load Balancer, with Cloud CDN

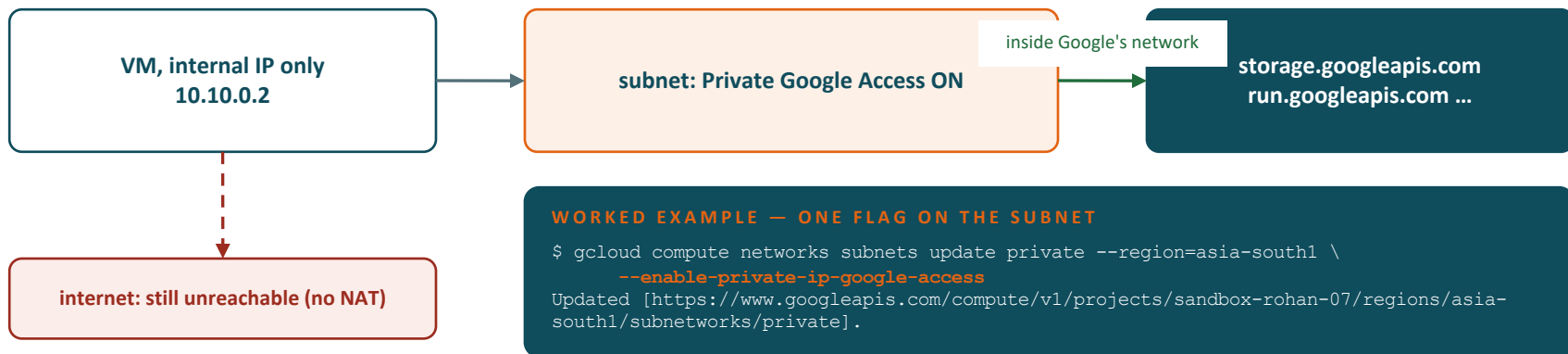


Add-on	Where	Gives you
Cloud CDN	per backend	edge caching of cacheable responses; cache keys, TTL overrides, invalidation
Cloud Armor	at the proxy	WAF rules (OWASP), rate limiting, IP/geo allow/deny
Identity-Aware Proxy	at the proxy	Google-identity sign-in in front of internal tools — no VPN

THE RULE

One IP, one certificate, one URL map — and the cache, the WAF and the sign-in page are checkboxes on it, not servers you run.

Private Google Access: reach Google APIs with no external IP

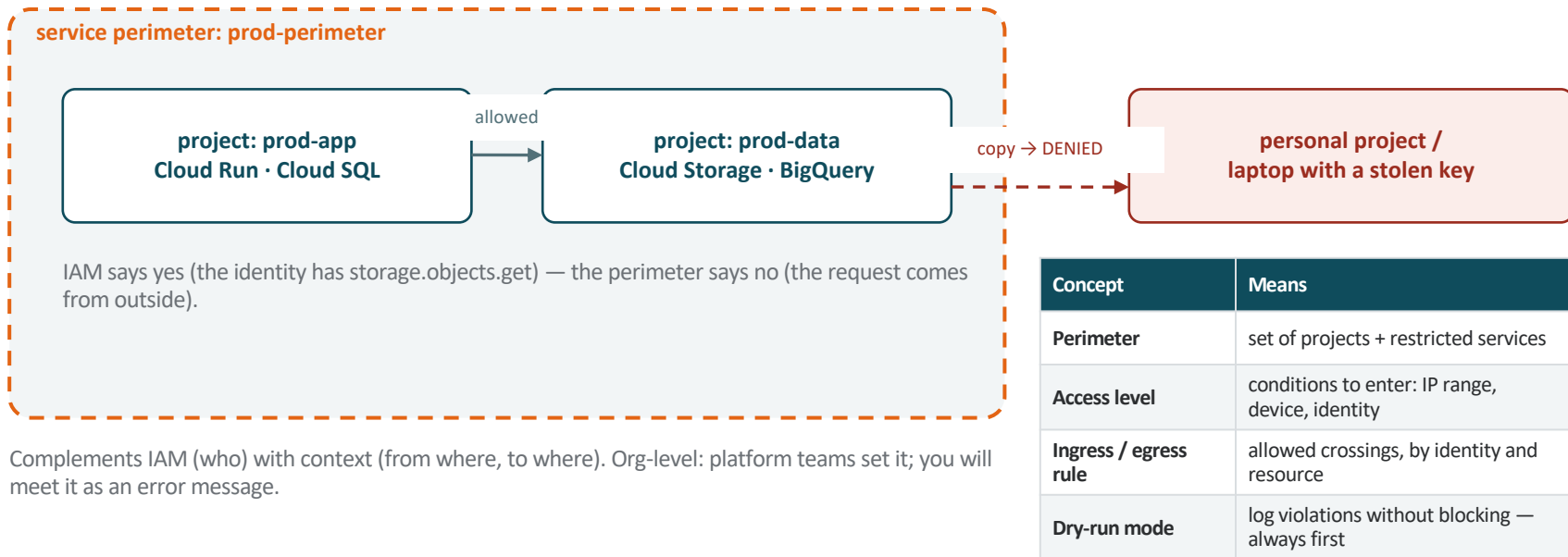


Option	What it is	When
Private Google Access	subnet flag; default route; public API IPs delivered privately	the default choice — Lab 4
private/restricted.googleapis.com	fixed VIPs + private DNS zone; restricted = VPC-SC-supported APIs only	no default route, or perimeter enforcement
Private Service Connect	an endpoint IP in your subnet for Google APIs or a producer's service	hybrid, strict egress, partner services

THE RULE

Turn Private Google Access on for every subnet with internal-only VMs — it is free, and it is the difference between "unreachable" and "works".

VPC Service Controls: a perimeter around the data, on top of IAM



Complements IAM (who) with context (from where, to where). Org-level: platform teams set it; you will meet it as an error message.

THE RULE

IAM decides who; a VPC Service Controls perimeter decides from where and to where — you need both to stop a stolen key.

Cloud DNS zones, and the four ways a service is allowed to be reached

WORKED EXAMPLE — A PUBLIC ZONE, AN A RECORD, A PRIVATE ZONE

```
$ gcloud dns managed-zones create example-public --dns-name=example.com. --description="public zone"
Created [https://dns.googleapis.com/dns/v1/projects/sandbox-rohan-07/managedZones/example-public].
$ gcloud dns record-sets create api.example.com. --zone=example-public --type=A --ttl=300 --rrdatas=34.120.45.67
NAME                TYPE  TTL  DATA
api.example.com.    A      300  34.120.45.67
$ gcloud dns managed-zones create corp-internal --dns-name=internal.example. --visibility=private --networks=prod-vpc
```

Exposure	How	Use for
Public, anonymous	Cloud Run --allow-unauthenticated / LB with no auth	marketing site, public API — deliberately
Public, protected	global external ALB + Cloud Armor (WAF, rate limits) + TLS	customer-facing apps
Identity at the edge	Identity-Aware Proxy on the LB or Cloud Run	internal tools: no VPN, sign in with Google identity
Internal only	Cloud Run --ingress=internal · internal LB · no external IP	backends called only by other services

THE RULE

Give every service the narrowest rung of the ladder that still lets its real callers in — and write which rung it is on.

YOUR CALL

A VM with no external IP runs `pip install` and `gcloud storage ls` — what happens?

Subnet: custom VPC, Private Google Access OFF, no Cloud NAT. Both commands time out. You turn Private Google Access ON. Now?

A) Both work

PGA gives the VM a way out.

B) Neither works

Only an external IP or NAT gives a way out.

C) gcloud storage works,

pip does not

PGA covers Google APIs only; PyPI is the internet.

D) pip works, gcloud

storage does not

PGA is for package mirrors.

A private subnet that can still reach Google

HANDS-ON · LAB 4

1

Step 1 — create a custom-mode VPC bookshelf-vpc

No subnets until you say so

2

TODO 4.2 — one subnet in asia-south1 with a /24 inside 10.0.0.0/8

How many usable addresses is that?

3

TODO 4.3 — firewall: SSH (tcp:22) only from 35.235.240.0/20 (IAP) to tag private-vm

Nothing else is open on this network

4

Step 4 — list the routes the VPC got for free

One local route per subnet, one default route to the internet gateway

5

TODO 4.5 — a VM with NO external IP; Step 6 — SSH in through IAP

gcloud compute ssh --tunnel-through-iap runs two curls: internet and the Storage API both fail

6

TODO 4.7 — turn on Private Google Access; Step 8 — rerun the curls; teardown

APIs now answer, the internet still does not

THE BLANKS 4.2 · 4.3 · 4.5 · 4.7

```
--range=_____  
  
--rules=tcp:_____ \  
--source-ranges=_____  
  
--subnet=bookshelf-private _____  
  
subnets update ... _____
```

Answers on the next slide. Console path: VPC network → Create VPC (custom); Firewall → Create rule; VM → Create (Networking: no external IP); subnet → Edit → Private Google Access On.

THE RULE

Custom VPC, narrow firewall, no external IP, Private Google Access — that is the shape of every production subnet.

What you should have seen — and the four answers

DEBRIEF · LAB 4

ANSWERS 4.2 · 4.3 · 4.5 · 4.7

```
--range=10.10.0.0/24          # 252 usable  
--rules=tcp:22 --source-ranges=35.235.240.0/20  
--no-address  
--enable-private-ip-google-access
```

TAKE WITH YOU

- No external IP + PGA + Cloud NAT (if needed) = the production subnet shape
- IAP TCP forwarding replaces bastion hosts

EXPECTED — STEPS 6 AND 8

```
# step 6 (before PGA)  
internet: unreachable (expected)  
storage api: unreachable (expected)  
# step 8 (after PGA)  
internet: unreachable (expected)  
storage api: HTTP 400  
gs://bookshelf-docs-sandbox-rohan-07/
```

COMMON STUMBLER

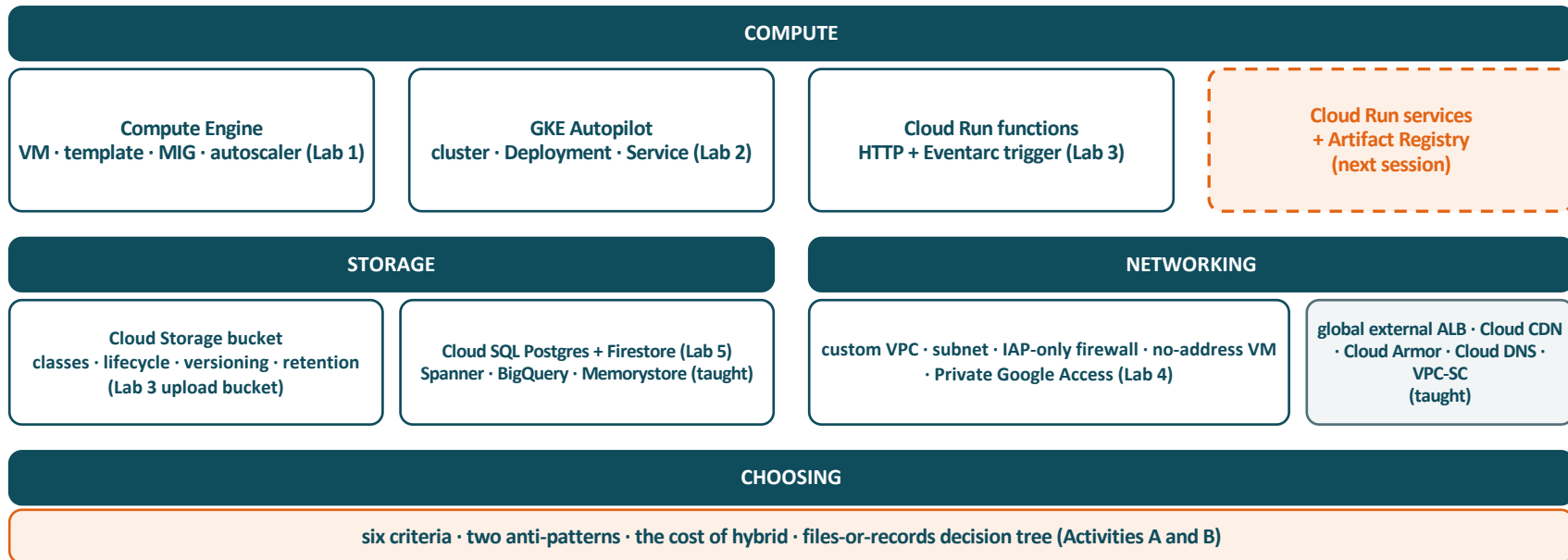
- SSH hangs: the IAP firewall rule is missing or has the wrong range
- Subnet delete fails: the VM is still there — teardown order

THE RULE

HTTP 400 from a Google API was the best news of the lab — the private path works.

THE LANDSCAPE IN ONE PICTURE

Where today's five labs landed — and what the next session adds



BEFORE YOU LEAVE

Tear it all down — and what it would have cost to forget

1

Lab 2 — delete the cluster (the expensive one)

kubectl delete service hello-web · gcloud container clusters delete hello-cluster --location asia-south1 --quiet

2

Lab 5 — delete the Cloud SQL instance and the Firestore database

gcloud sql instances delete bookshelf-pg --quiet · gcloud firestore databases delete --database='(default)' --quiet

3

Lab 3 — delete the trigger, the two functions and the bucket

gcloud eventarc triggers delete on-upload ... · gcloud run services delete hello-function / gcs-function · gcloud storage rm -r gs://bookshelf-uploads-<project>

4

Lab 4 — VM → firewall rule → subnet → network

bash teardown.sh in lab4 (order matters)

5

Lab 1 — if you skipped it earlier

bash teardown.sh in lab1

6

Verify

gcloud compute instances list · gcloud container clusters list · gcloud run services list · gcloud sql instances list · gcloud compute networks list → empty (only 'default' remains)

Left running overnight	Roughly
3 small VMs (Lab 1)	≈ US\$1 / day
an idle GKE Autopilot cluster	≈ US\$2–3 / day
a db-f1-micro Cloud SQL instance	≈ US\$0.35 / day
functions at zero traffic	0
a bucket with a few KB	≈ 0

Prices are order-of-magnitude, on the build date, and change — the habit does not.

THE RULE

A lab is finished when the resource list is empty — not when the page loads.

WHAT COMES NEXT

You have the map. The next session containerizes the service and lands it on Cloud Run.

- Compute: the container rung you skipped today — Dockerfile, image, Artifact Registry, Cloud Run sizing, revisions.
- Storage: the Postgres table and the Firestore documents from Lab 5 are what that service will read.
- Networking: the private path Lab 4 proved is how a Cloud Run service reaches a private backend.
- Choosing: the six criteria decided Cloud Run for the two-service app — the next session shows why in practice.